Architettura Software, Modellazione UML e Teoria OOP - Albero di Regressione

Architettura Software, Modellazione UML e Fondamenti di Teoria OOP

Progetto: Albero di Regressione (Modulo Server e Client)
Corso Accademico: Metodi Avanzati di Programmazione (MAP) — Dipartimento di Informatica, Università degli Studi di Bari Aldo Moro
Docenti di Riferimento: Prof. Donato Malerba, Prof.ssa Annalisa Appice, Prof. Michelangelo Ceci
Autore / Branch: Teamwork Engineering Agent (Branch: `docs/uml-architettura-oop`)
Data: Anno Accademico 2025/2026 — JDK 25

Sommario Esecutivo

Il presente documento formalizza l'architettura software, la modellazione a oggetti e i fondamenti di Ingegneria del Software del sistema **Albero di Regressione**, correlandoli sistematicamente ai principi teorici avanzati del corso *Metodi Avanzati di Programmazione*. Vengono analizzati in profondità: 1. **La Teoria dell'Astrazione**: l'astrazione come processo cognitivo ed entità concettuale, la dicotomia tra specifica (*what*) e realizzazione (*how*), i ruoli di utente e implementatore, la cittadinanza dei valori, e la declinazione delle **6 classi sintattiche** di linguaggi di programmazione applicate alla codebase Java del progetto. 2. **Information Hiding e Pareti Concettuali**: le tre regole dell'astrazione dati, l'incapsulamento come barriera protettiva e la tassonomia delle pareti (*opache*, *traslucide*, *trasparenti*). 3. **Il Package come Modulo**: il parallelismo con i moduli del linguaggio Ada (Package Specification vs. Package Body), la giustificazione architetturale della visibilità package-private nel package `tree`, e l'analisi critica della violazione dell'*Acyclic Dependencies Principle* (ADP) tra `tree` e `server`. 4. **La Matrice di William R. Cook (1991)**: la distinzione fondamentale tra Tipi Astratti di Dato (ADT) e Classi di Oggetti, con dimostrazione teorica del perché le gerarchie `Node` e `Attribute` adottano il paradigma a oggetti (ereditarietà per estensione e variazione, Liskov Substitution Principle, dynamic binding, RTTI). 5. **Specifiche Sintattiche, Semantiche (Pre/Post) e Tabelle Algebriche**: formalizzazione assiomatica rigorosa e matrici Costruttori × Osservazioni per `Example`, `Data`, `SplitNode` e `RegressionTree`. 6. **Modellazione UML Formale**: diagrammi completi e rigorosi in sintassi Mermaid.js per i Package (con relazioni `«import»` non transitive), le Classi (con visibilità completa, classi astratte, membri statici, classe attiva `ServerOneClient` e inner class `SplitInfo`), la Sequenza del protocollo socket Client-Server e il Deployment fisico Three-Tier.

---

1. Fondamenti Teorici di Astrazione applicati al Progetto

1.1 Astrazione come Processo e come Entità

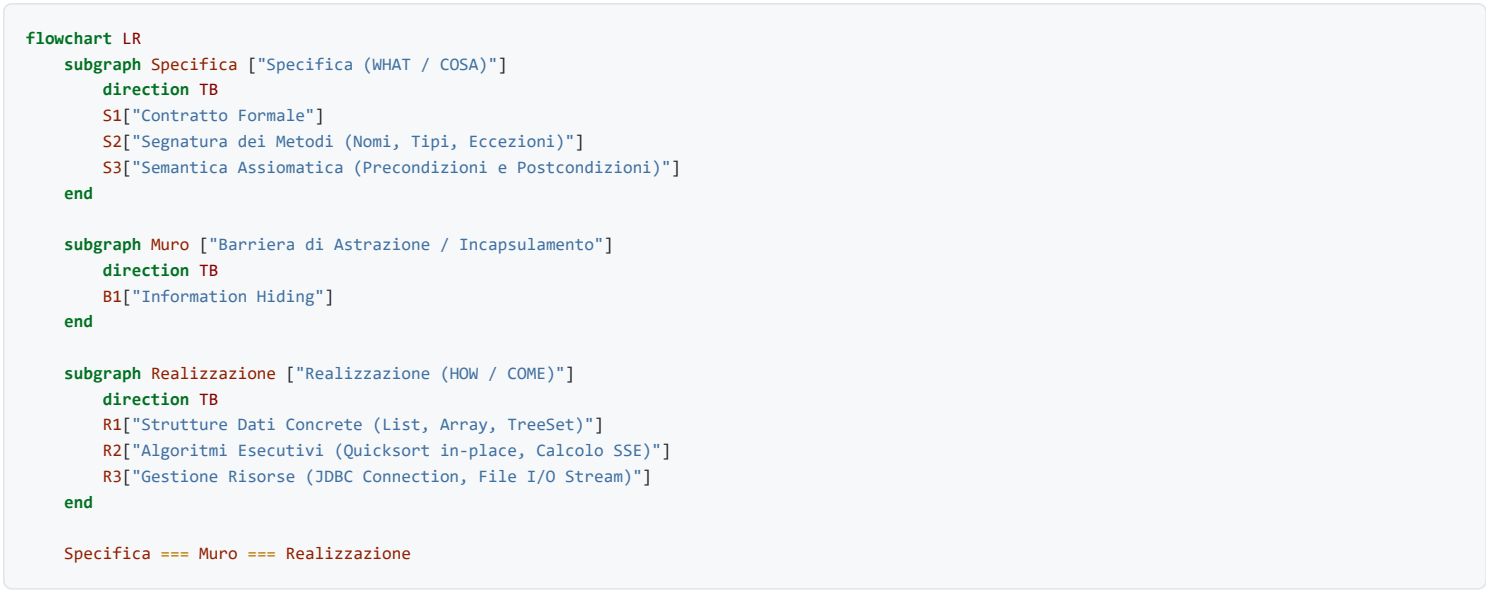
Nella teoria dei linguaggi e dell'Ingegneria del Software, il termine **astrazione** (dal latino *abstrahere*, *"trarre da"*, *"isolare"*) denota un principio duale: - **Come Processo cognitivo**: è l'operazione intellettuale mediante la quale si isolano gli attributi essenziali e invarianti di un dominio o di un sottosistema, trascurando deliberatamente tutti i dettagli fisici, contingenti, accidentali o implementativi che non influiscono sullo scopo prefissato. - **Come Entità concettuale**: è il manufatto risultante (un'interfaccia, una classe, un tipo di dato, una

segnatura) che esprime un modello semplificato ma rigoroso, comprensibile e manipolabile senza richiedere la conoscenza dei meccanismi interni di realizzazione.

Nel sistema dell'Albero di Regressione, tale principio si manifesta a tutti i livelli: - **Astrazione sui Dati Tabellari (`Data`)**: I dati grezzi memorizzati su DBMS relazionale MySQL sotto forma di pagine disco InnoDB, indici B-Tree e lock transazionali vengono trasfigurati nell'entità astratta `Data` . Essa espone esclusivamente un'algebra di tuple di campioni (`Example`), un insieme ordinato di attributi indipendenti continui o discreti (`explanatorySet`), e una variabile dipendente numerica continua (`classAttribute`). - **Astrazione sul Modello di Regressione (`RegressionTree`)**: Il processo di regressione simbolico-numerica viene reificato in una struttura gerarchica ad albero che incapsula la funzione predittiva $\hat{y} = f(\boldsymbol{x})$. Il fruitore interagisce con l'albero come decisore oracolare tramite il metodo `predictClass(...)` , rimanendo completamente all'oscuro della complessità combinatoria dell'induzione top-down e dell'allocazione dei nodi nello Heap.

1.2 Specifica (Cosa) vs Realizzazione (Come)

La separazione tra **specifica** e **realizzazione** costituisce la pietra angolare per il controllo della complessità e per la manutenibilità del software nel lungo termine:



- **La Specifica (*What*)**: Definisce il contratto visibile verso l'esterno. Stabilisce *quali* operazioni sono consentite, quali argomenti richiedono, quali eccezioni controllate possono sollevare, e quali garanzie logico-matematiche assicurano all'uscita (postcondizione), assumendo che l'ambiente rispetti le precondizioni stabilite.
- **La Realizzazione (*How*)**: Rappresenta il codice privato e le strutture dati interne scelte dall'ingegnere del software per adempiere al contratto promesso.

Questa bipartizione sancisce i ruoli operativi nel ciclo di vita del software: - **I Fruitori (Utenti del modulo)**: Si limitano a comprendere e rispettare la specifica. Ad esempio, la classe `ServerOneClient` invoca `tree.predictClass(out, in)` conoscendo unicamente il protocollo interattivo di predizione; non deve e non può preoccuparsi di come `SplitNode` ottimizzi il test o di quale sia la varianza dei singoli nodi interni. - **Gli Implementatori**: Sono liberi di riprogettare, profilare e ottimizzare l'algoritmo interno (ad esempio, sostituendo l'algoritmo di split Quicksort con un algoritmo intro-sort $O(N \log N)$ garantito nel caso pessimo, o memorizzando l'albero in vettori contigui per massimizzare la cache-locality della CPU), con la certezza matematica che **nessuna linea di codice fruitore dovrà essere modificata o ricompilata**, purché la specifica rimanga immutata.

1.3 Cittadinanza dei Valori (Denotabilità, Esprimibilità, Memorizzabilità)

Secondo la classificazione formale dei linguaggi di programmazione (proposta da Strachey e formalizzata nella teoria dei linguaggi), lo status di un valore computazionale si misura su tre assiomi: 1. **Denotabile**: Se il valore può essere associato a un identificatore o nome simbolico nel programma (es. variabili locali, parametri formali). 2. **Esprimibile**: Se il valore può essere il risultato della valutazione di un'espressione complessa senza dover passare per un'assegnazione intermedia. 3. **Memorizzabile**: Se il valore può essere immagazzinato all'interno di una locazione di memoria variabile (campo di classe o cella di array).

Un'entità che soddisfa contemporaneamente tutte e tre le proprietà è definita **Cittadino di Prima Classe** (*First-Class Citizen*).
Nel nostro progetto Java: - Gli oggetti `Example`, `Data`, `RegressionTree`, `Node` sono **cittadini di prima classe**: possono essere denotati con variabili, prodotti come risultato immediato di espressioni (es. `new RegressionTree(new Data("provaC"))`), memorizzati in collezioni (`List<Example> data`), e serializzati su stream di byte. - Le computazioni stesse (tramite classi anonime, lambda expression e il reference di metodo di Java 8+, ad es. `Comparator.comparingDouble(...)`) sono promosse a cittadini di prima classe.

1.4 Le 6 Classi Sintattiche di Astrazione

Il principio cardine enunciato da Malcolm Cowper stabilisce che: > *"È possibile costruire astrazioni su una qualunque classe sintattica, purché le frasi di quella classe specifichino un qualche tipo di computazione."*

Se una frase sintattica non specifica una computazione, l'astrazione non è concettualmente applicabile.
Controesempio: I **letterali** (es. il numero `42`, la stringa `"map"`, il carattere `'c'`). Essi rappresentano costanti statiche immediate; non descrivono alcun processo di calcolo o trasformazione, e pertanto non ammettono astrazione.

La tabella seguente mappa rigorosamente le sei classi sintattiche sui costrutti del progetto:

Classe Sintattica	Meccanismo di Astrazione	Computazione Sottintesa	Manifestazione Esatta nella Codebase del Progetto
Espressione	**Astrazione di Funzione**	Valutazione pura di un'espressione che calcola e restituisce un valore, idealmente senza alterare lo stato (funzione priva di side-effect).	**Metodi osservatori puri**: • `Node.getVariance()` : calcola o restituisce la devianza del nodo. • `Example.get(int i)` : valuta e restituisce il valore associato alla *i*-esima colonna. • `Attribute.getName()` : restituisce il nome simbolico dell'attributo. • `DiscreteNode.getNumberOfChildren()` : restituisce la cardinalità delle partizioni discrete discrete. • `TableSchema.getNumberOfAttributes()` : restituisce il conteggio delle colonne.
Comando	**Astrazione di Procedura**	Esecuzione di una sequenza di comandi che alterano lo stato della memoria o del sistema operativo (mutazione di stato con effetti collaterali controllati).	**Metodi modificatori (mutatori)**: • `Data.sort(Attribute, int, int)` : permuta in-place il vettore degli esempi per intervalli contigui. • `Example.add(Object o)` : muta la lista interna accodando un nuovo valore. • `RegressionTree.learnTree(...)` : popola ricorsivamente i campi dell'albero alterando la memoria Heap. • `DbAccess.initConnection()` : alloca la connessione TCP/IP nativa con il server MySQL.
Controllo di sequenza	**Astrazione di Controllo**	Meccanismi sintattici che disciplinano l'ordine di valutazione, la ramificazione, l'iterazione, la ricorsione e la concorrenza.	**Quattro forme di controllo astratto**: 1. *Iterazione Polimorfa*: `DiscreteAttribute implements Iterable<String>` , `TableSchema implements Iterable<Column>` , `Example implements Iterable<Object>` . Permette la scansione con costrutto astratto `for-each` senza esporre indici di scorrimento né strutture dati di contenimento. 2. *Ricorsione Strutturale*: `quicksort(...)` in `Data` , `learnTree(...)` in `RegressionTree` , discesa ricorsiva in `predictClass(...)` , visita in `printRules(String)` . 3. *Gestione delle Anomalie*: costrutti strutturati `try-catch-finally` con gerarchie di eccezioni tipizzate a

Classe Sintattica	Meccanismo di Astrazione	Computazione Sottintesa	Manifestazione Esatta nella Codebase del Progetto
			terminazione (`TrainingDataException` , `DatabaseConnectionException` , `EmptySetException` , `UnknownValueException`). 4. *Concorrenza*: il thread attivo `ServerOneClient extends Thread` astrae il flusso concorrente del server socket multi-client.
Accesso alla memoria	**Astrazione di Selettore**	Calcolo dell'indirizzo (L-value) o mediazione protetta nell'accesso a una cella di memoria o campo di un aggregato.	In Java non esistono puntatori espliciti né riferimenti modificabili di tipo L-value come in C++ (`int&`). L'astrazione di selettore viene ottenuta incapsulando la locazione interna tramite metodi getter/setter con validazione d'indice: `Example.get(int i)` , `Data.getExplanatoryValue(int i, int j)` , `TableSchema.getColumn(int index)` .
Definizione di un dato	**Astrazione di Tipo**	Definizione di un dominio algebrico di valori correlato all'insieme chiuso delle operazioni ammissibili, prescindendo dalla codifica a bit.	**Classi di dominio come Tipi Astratti di Dato (ADT)**: • `Data` : astrae una matrice di dati di addestramento. • `RegressionTree` : astrae un modello predittivo continuo strutturato ad albero. • `ContinuousAttribute` e `DiscreteAttribute` : astraggono i domini di variabili numeriche reali e categoriche finite.
Dichiarazione	**Astrazione Generica**	Elaborazione di una dichiarazione parametrica o matrice che funge da modello per la produzione di legami (*bindings*) per istanziazione.	**Due livelli di astrazione generica**: 1. *Java Generics (Polimorfismo Parametrico)*: `TreeSet<SplitNode>` , `List<Example>` , `List<SplitInfo>` , `Comparable<SplitNode>` , `Iterable<Object>` . La logica dell'algoritmo di collezione o ordinamento è scritta una volta sola e specializzata a compile-time garantendo il controllo statico dei tipi con type erasure a runtime. 2. *La Classe come Modello Generico di Istanze*: Ogni dichiarazione di classe funge da schema generatore di moduli dotati di stato locale incapsulato e

Classe Sintattica	Meccanismo di Astrazione	Computazione Sottintesa	Manifestazione Esatta nella Codebase del Progetto
			costruttore parametrico (`new DiscreteNode(...)`).

---

2. Information Hiding, Incapsulamento e Pareti Concettuali

2.1 I Principi Fondamentali di David Parnas

Il principio dell'**Information Hiding** (David Parnas, 1972) stabilisce che un modulo software ben progettato deve celare al proprio esterno due categorie di informazioni: 1. I dettagli tecnologici di basso livello soggetti a frequente obsolescenza o mutamento (es. stringhe SQL, formati binari di memorizzazione). 2. Le scelte progettuali arbitrarie interne (es. scelta di un array dinamico rispetto a una lista doppiamente concatenata).

L'**Incapsulamento** è il meccanismo linguistico che realizza concretamente l'information hiding, racchiudendo entro una capsula sintattica coesa le variabili di stato e i metodi che vi operano, presidiandola con modificatori di visibilità.

Dalla combinazione di questi principi discendono le **3 regole d'oro dell'astrazione sui dati**: 1. **Regola 1 (Privatizzazione della rappresentazione)**: Tutti i campi di stato interno devono essere dichiarati con visibilità restrittiva (`private`), precludendo l'accesso e la mutazione diretta dall'esterno. 2. **Regola 2 (Mediazione tramite operazioni lecite)**: Qualunque lettura, mutazione o transizione di stato deve passare obbligatoriamente attraverso le operazioni pubbliche o protette definite nel contratto d'interfaccia, garantendo l'invariante di classe. 3. **Regola 3 (Impacchettamento unitario)**: La rappresentazione dello stato e l'insieme delle operazioni lecite devono risiedere all'interno della medesima unità sintattica compilabile (la classe).

2.2 Tassonomia delle Pareti Concettuali nel Progetto

```mermaid
graph TD
    subgraph Parete_Opaca ["PARETE OPACA (Massimo Occultamento)"]
        direction TB
        PO_Ext["Moduli Esterni / Client"] -->|"Solo Interfaccia Pubblica (+)"| PO_Box["RegressionTree / DbAccess"]
        PO_Box ---|"Inaccessibile (-)"| PO_Int["Puntatori Node, Algoritmo SSE, JDBC Connection, Credenziali"]
    end

    subgraph Parete_Traslucida ["PARETE TRASLUCIDA (Visibilità Selettiva Protetta)"]
        direction TB
        PT_Ext["Classi Esterne al Package"] -.x|"Bloccato (No public)"| PT_Box["Node, SplitNode, LeafNode"]
        PT_Peer["Classi del Package tree (Peer)"] ==>|"Accesso Package (~) e Protected (#)"| PT_Box
    end

    subgraph Parete_Trasparente ["PARETE TRASPARENTE (Puro Trasporto Dati)"]
        direction TB
        PTR_User["SplitNode"] -->|"Ispezione Diretta Campi e Getter Immediati"| PTR_Box["SplitInfo (Inner Class)"]
    end

    style Parete_Opaca fill:#ffebee,stroke:#c62828,stroke-width:2px
    style Parete_Traslucida fill:#fff8e1,stroke:#f57f17,stroke-width:2px
    style Parete_Trasparente fill:#e8f5e9,stroke:#2e7d32,stroke-width:2px
```

1. Pareti Opache (Black-Box Isolation)

Le pareti opache realizzano il massimo grado di occultamento. Verso l'esterno non traspare alcun dettaglio della struttura sottostante: - **`RegressionTree`** : Costituisce la facciata opaca del modulo di regressione. I client (`ServerOneClient` , processi esterni) vedono unicamente i

metodi pubblici `predictClass(out, in)`, `printRules()`, `salva(nomeFile)` e `carica(nomeFile)`. È impossibile dall'esterno conoscere se l'albero utilizzi nodi continui o discreti, quale sia la struttura del puntatore alla radice `root`, o quali siano gli indici degli esempi. - **`DbAccess`**: Isola ermeticamente l'accesso al DBMS. Campi sensibili come `conn`, `DATABASE`, `USER_ID` e `PASSWORD` sono privati e immutabili. Il fruitore può solo richiedere l'apertura (`initConnection()`) o la chiusura (`closeConnection()`).

2. Pareti Traslucide (Package-Private & Protected Cooperation)

Una parete traslucida consente il passaggio della visibilità esclusivamente a un gruppo selezionato di moduli cooperanti (lo stesso package o sottoclassi derivate), agendo invece come una parete opaca invalicabile per il resto del mondo software: - **Gerarchia `Node`, `SplitNode`, `LeafNode`, `DiscreteNode`, `ContinuousNode`**: All'interno del package `tree`, la parete è traslucida: `RegressionTree` accede ai metodi package-private `~getVariance()`, `~getNumberOfChildren()`, `~getIdNode()`. Nelle classi derivate, `mapSplit` è dichiarato `protected` (`#`) consentendo a `DiscreteNode` e `ContinuousNode` di popolare direttamente le partizioni figlie.
Tuttavia, verso qualsiasi classe esterna al package `tree`, la parete diviene **totalmente opaca**: nessuna classe esterna può istanziare, ereditare o invocare un `Node`.

3. Pareti Trasparenti (White-Box / Data Transfer Carriers)

Le pareti trasparenti non impongono barriere comportamentali complesse, poiché l'oggetto ha la funzione di mero aggregatore strutturato di valori: - **Inner Class `SplitNode.SplitInfo`**: Incapsula una quintupla di valori elementari (`splitValue`, `beginIndex`, `endIndex`, `numberChild`, `comparator`). Non possiede invarianti di stato articolate né logica computazionale; funge da struttura di trasporto dati interna al servizio esclusivo di `SplitNode`.

---

3. Il Costrutto `package` come Modulo (Specification vs Body)

3.1 Parallelismo con i Moduli Ada e Modula-2

Nell'Ingegneria del Software classica, il linguaggio **Ada** (sviluppato su specifiche del Dipartimento della Difesa USA per garantire massima affidabilità) ha formalizzato la struttura del modulo attraverso il costrutto `package`, nettamente ripartito in due parti fisiche e concettuali: 1. **Package Specification (`.ads`)**: Contiene la dichiarazione formale delle entità esportate (tipi, costanti, firme di sottoprogrammi). Essa include una parte pubblica visibile a tutti i client compilatori e, opzionalmente, una sezione `private` in cui sono dichiarati i dettagli di implementazione dei tipi privati astratti (necessari al compilatore per calcolare la dimensione di allocazione dello stack, pur rimanendo inaccessibili al programmatore fruitore). 2. **Package Body (`.adb`)**: Contiene la realizzazione algoritmica dei sottoprogrammi e le variabili interne di modulo. Il body è completamente nascosto e può essere ricompilato indipendentemente dalla specification.

In **Java**, questa medesima architettura modulare viene realizzata mediante la combinazione della direttiva `package` con i modificatori di accesso: - La **Package Specification** esportata verso l'esterno è costituita esclusivamente dalle classi e interfacce dichiarate con visibilità **`public` (`+`)**. - Il **Package Body** privato è costituito da tutte le classi, interfacce, campi e metodi aventi **visibilità package-private (`~`)** (assenza di modificatore).

```mermaid
flowchart TB
    subgraph Ada_Module ["Modulo Ada"]
        direction TB
        AS["Package Specification (.ads)<br/>• public spec: tipi, costanti, segnature<br/>• private spec: tipi privati"]
        AB["Package Body (.adb)<br/>• Implementazione algoritmi<br/>• Variabili e procedure locali nascoste"]
        AS -.->|"Implementato da"| AB
    end

    subgraph Java_Package ["Package Java (es. package tree)"]
        direction TB
        JS["Package Specification (Esportata)<br/>• public class RegressionTree<br/>• Metodi pubblici (+printRules, +predictClass)"]
        JB["Package Body (Incapsulato Internamente)<br/>• abstract class Node (~)<br/>• abstract class SplitNode (~)<br/>• class LeafNode (~)<br/>• class DiscreteNode (~)<br/>• class ContinuousNode (~)"]
        JS -.->|"Composto internamente da"| JB
    end
```

3.2 Giustificazione Architetturale nel Package `tree`

Nel package `tree` del nostro progetto: - `RegressionTree` è l'**unica classe** `public`. - `Node`, `SplitNode`, `LeafNode`, `DiscreteNode`, `ContinuousNode` e l'inner class `SplitInfo` sono **package-private (`~`)**.

Motivazione Ingegneristica: 1. **Minimizzazione della Superficie di Attacco e di Accoppiamento**: Se le classi dei singoli nodi fossero dichiarate `public`, gli sviluppatori di altri package potrebbero istanziare direttamente nodi orfani, tentare manipolazioni manuali della struttura ad albero o creare dipendenze dirette con la logica interna dei nodi. 2. **Libertà di Riorganizzazione Interna (Invarianza ai Cambiamenti)**: Mantenendo i nodi confinati nel *Body* del package, l'implementatore può modificare in qualsiasi momento la gerarchia interna (ad es. introducendo un nodo per regressione polinomiale o trasformando l'albero in un grafo aciclico orientato DAG) senza che alcun modulo esterno ne risenta o debba essere ricompilato.

3.3 Analisi Critica dell'Accoppiamento Ciclico (`tree` ↔ `server`) e Violazione ADP

Un'indagine rigorosa delle direttive `import` evidenzia una violazione dell'**Acyclic Dependencies Principle (ADP)** (Robert C. Martin): - La classe `ServerOneClient` (nel package `server`) esegue l'importazione di `tree.RegressionTree` per istanziare e addestrare il modello. - La classe `RegressionTree` (nel package `tree`) esegue l'importazione di `server.UnknownValueException` nella segnatura del metodo `predictClass(out, in)`: `java public Double predictClass(ObjectOutputStream out, ObjectInputStream in) throws IOException, ClassNotFoundException, UnknownValueException`

```mermaid
flowchart LR
    P_SERVER["Package server<br/>(ServerOneClient, MultiServer)"]
    P_TREE["Package tree<br/>(RegressionTree, Node, ...)"]

    P_SERVER -->|"import tree.RegressionTree<br/>(Uso del Modello)"| P_TREE
    P_TREE -.->|"import server.UnknownValueException<br/>(Segnatura predictClass)"| P_SERVER

    style P_SERVER fill:#ffcdd2,stroke:#b71c1c,stroke-width:2px
    style P_TREE fill:#ffcdd2,stroke:#b71c1c,stroke-width:2px
```

Diagnosi Ingegneristica e Soluzione: Secondo i canoni dell'Architettura Esagonale (*Ports and Adapters*) e della *Clean Architecture*, il Core Domain (`tree`) rappresenta la logica di business fondamentale e non deve **mai** dipendere dai moduli infrastrutturali o di trasporto di rete (`server`).
L'eccezione `UnknownValueException` rappresenta l'evento semantico in cui un valore da predire non trova corrispondenza in alcun ramo di split dell'albero: appartiene quindi a pieno titolo al dominio concettuale del package `tree`.
Risoluzione raccomandata: Spostare `UnknownValueException` all'interno del package `tree` (o in un package neutro trasversale `exception`), eliminando definitivamente la freccia ciclica da `tree` a `server` e ristabilendo la struttura a grafo aciclico orientato (DAG) delle dipendenze.

---

4. Matrice di W.R. Cook (1991): Tipi Astratti di Dato (ADT) vs Classi di Oggetti

Nel saggio fondamentale *Object-Oriented Programming Versus Abstract Data Types* (Foundations of Object-Oriented Languages, 1991), William R. Cook ha dimostrato formalmente che ADT e Programmazione a Oggetti (OOP) sono due paradigmi ortogonali e complementari di astrazione sui dati, spesso confusi:

Dimensione di Analisi	Tipo Astratto di Dato (ADT)	Classe di Oggetti (OOP)	Riscontro nel Progetto Albero di Regressione
Principio Organizzativo	Organizzato attorno alle **Osservazioni / Operazioni**. Ogni operazione è una funzione centrale che discrimina tutti i casi con costrutti condizionali (`switch` / `if-else`).	Organizzata attorno ai **Costruttori / Varianti di Tipo**. Ogni costruttore è una classe autonoma che implementa tutte le operazioni su di sé.	Nella gerarchia `Node` , ciascun costruttore (`LeafNode` , `DiscreteNode` , `ContinuousNode`) è una classe distinta che fornisce la propria implementazione delle osservazioni (`getVariance()` , `getNumberOfChildren()`).
Fondamento Matematico	**Algebra Eterogenea**: un insieme chiuso di valori manipolabile esclusivamente attraverso una segnatura globale di funzioni esterne.	**Interfaccia Comportamentale**: un insieme di operazioni legate dinamicamente al contesto implicito dell'oggetto ricevitore (`this`).	`Node` e `Attribute` non passano se stessi come strutture passive a procedure esterne, ma rispondono polimorficamente a messaggi: `node.testCondition(val)` .
Segnatura Sintattica	Operazioni con dato esplicito come parametro formale: $\text{testCondition}(node, val)$.	Operazioni con dato implicito come ricevitore del messaggio: $node.\text{testCondition}(val)$.	Invocazione a oggetti: `splitNode.testCondition(val)` .
Compilazione del Codice	Il codice di ciascuna operazione è scritto e compilato **una sola volta** in un unico modulo procedurale.	Il codice dei metodi è compilato **distribuito per ogni classe derivata** della gerarchia.	Ogni sottoclasse di `SplitNode` compila il proprio specifico algoritmo di partizionamento `setSplitInfo(...)` .
Estendibilità sui Costruttori (Aggiunta Nuove Varianti)	**Difficile (Chiusura sui Costruttori)**: L'aggiunta di una nuova variante di dato impone la riapertura, riscrittura e ricompilazione intrusiva di **tutte** le funzioni dell'ADT.	**Facile (Apertura sui Costruttori)**: L'aggiunta di una nuova classe richiede unicamente la scrittura della nuova sottoclasse, senza alterare né ricompilare il codice esistente.	**Motivo cardine dell'adozione OOP per i Nodi**: Aggiungere un `MultiWayContinuousNode` o un `FuzzySplitNode` non tocca minimamente `LeafNode` o `RegressionTree` !
Estendibilità sulle Operazioni (Aggiunta Nuove Funzioni)	**Facile (Apertura sulle Operazioni)**: Aggiungere una nuova operazione richiede solo la scrittura di una nuova funzione autonoma, senza toccare i costruttori esistenti.	**Difficile (Chiusura sulle Operazioni)**: Aggiungere un nuovo metodo impone la modifica della classe/interfaccia base e di tutte le sottoclassi concrete della gerarchia.	L'aggiunta di un'operazione trasversale come `prune()` richiederebbe l'aggiornamento di `Node` e di tutte le sue sottoclassi (a meno di adottare il pattern *Visitor*).

4.1 Giustificazione OOP delle Gerarchie `Node` e `Attribute`

La scelta di modellare `Node` e `Attribute` secondo il paradigma OOP anziché procedurale risponde a quattro pilastri teorici:

1. **Ereditarietà per Estensione e per Variazione Funzionale**:
   - *Estensione di Stato*: `DiscreteAttribute` estende `Attribute` introducendo il set di valori ammissibili `Set<String> values`. `LeafNode` estende `Node` introducendo il valore numerico stimato `Double predictedClassValue`.
   - *Variazione Funzionale (Overriding)*: `SplitNode` sovrascrive il metodo `getVariance()` di `Node`. Mentre in `Node` la varianza rappresenta la devianza locale della partizione corrente, in `SplitNode` essa restituisce `splitVariance` (la somma pesata delle varianze dei rami figli generati dal test), consentendo a `RegressionTree` di calcolare istantaneamente il guadagno d'informazione o la riduzione d'errore senza conoscere la natura dello split.
2. **Polimorfismo di Inclusione e Principio di Sostituibilità di Liskov (LSP)**:
   - Secondo la tassonomia di Cardelli & Wegner (1985), il polimorfismo di inclusione consente a un'espressione di assumere tipi diversi purché legati da una relazione di sottotipizzazione. Nel nostro albero, il campo di radice è tipizzato come `Node root`.
   - In conformità con l'LSP formulato da Barbara Liskov (1987), qualsiasi istanza derivata (`LeafNode`, `DiscreteNode`, `ContinuousNode`) può essere sostituita a un riferimento di tipo `Node` senza alterare alcuna delle proprietà desiderabili del programma (correttezza, invarianza parziale, terminazione).
3. **Legame Dinamico (Late / Dynamic Binding)**:
   - Invocando `root.getNumberOfChildren()` o `root.toString()`, il compilatore inserisce l'istruzione bytecode `invokevirtual`. A tempo di esecuzione, la JVM esamina il descrittore del tipo concreto allocato nello Heap e consulta la relativa *Virtual Method Table* (vtable), risolvendo l'indirizzo esatto del metodo specifico del nodo in tempo costante $O(1)$.
4. **RTTI (Run-Time Type Information)**:
   - In precisi punti architetturali, il sistema ricorre al controllo dinamico dei tipi mediante l'operatore `instanceof`:
     - In `RegressionTree.determineBestSplitNode(...)`: discrimina la creazione di un `DiscreteNode` o di un `ContinuousNode` interrogando la natura dell'attributo in esame.
     - In `Data.quicksort(...)`: delega il partizionamento a `partition(DiscreteAttribute, ...)` o `partition(ContinuousAttribute, ...)` in base alla classe concreta dell'attributo.

---

5. Specifiche Sintattiche, Semantiche (Pre/Post) e Tabelle Costruttori × Osservazioni

Vengono formalizzate di seguito le specifiche algebriche e assiomatiche complete per le quattro classi cardine della logica di business.

5.1 Specifica Formale di `Example`

a) Specifica Sintattica (Sorts e Operazioni)

- **Sorts**: `Example`, `Object`, `Integer`, `Boolean`, `String`
- **Costruttori**:
  - $creaExample: \emptyset \rightarrow Example$ *(Costruttore base/generatore primitivo)*
  - $add: Example \times Object \rightarrow Example$ *(Costruttore generatore induttivo)*
- **Osservazioni**:
  - $get: Example \times Integer \rightarrow Object$
  - $size: Example \rightarrow Integer$
  - $isEmpty: Example \rightarrow Boolean$
  - $toString: Example \rightarrow String$

b) Specifica Semantica Assiomatica (Pre/Post Condizioni)

- $\boldsymbol{add(ex, o) \rightarrow ex'}$:
  - Precondizione: $ex \neq \text{null} \land o \neq \text{null}$
  - Postcondizione: $size(ex') = size(ex) + 1 \ \land \ get(ex', size(ex)) = o \ \land \ (\forall k \in [0, size(ex) - 1]: get(ex', k) = get(ex, k))$
- $\boldsymbol{get(ex, i) \rightarrow o}$:
  - Precondizione: $ex \neq \text{null} \land 0 \leq i < size(ex)$
  - Postcondizione: o è il valore della coordinata i-esima del campione.

c) Tabella Costruttori × Osservazioni per `Example`

Costruttori ↓ / Osservazioni →	$isEmpty(e)$	$size(e)$	$get(e, i)$
$creaExample()$	true	0	error (violazione precondizione: $i \notin [0, size - 1]$)
$add(ex, o)$	false	$size(ex) + 1$	if $i = size(ex)$ then o else $get(ex, i)$

5.2 Specifica Formale di `Data`

a) Specifica Sintattica (Sorts e Operazioni)

- **Sorts**: `Data`, `String`, `Integer`, `Double`, `Object`, `Attribute`, `ContinuousAttribute`
- **Costruttori**:
  - $caricaDaDB: String \rightarrow Data$ *(Inizializzatore tramite lettura da tabella relazionale)*
  - $sort: Data \times Attribute \times Integer \times Integer \rightarrow Data$ *(Riorganizzatore d'ordine interno)*
- **Osservazioni**:
  - $getNumberOfExamples: Data \rightarrow Integer$
  - $getNumberOfExplanatoryAttributes: Data \rightarrow Integer$
  - $getClassValue: Data \times Integer \rightarrow Double$
  - $getExplanatoryValue: Data \times Integer \times Integer \rightarrow Object$
  - $getClassAttribute: Data \rightarrow ContinuousAttribute$
  - $getExplanatoryAttribute: Data \times Integer \rightarrow Attribute$

b) Specifica Semantica Assiomatica (Pre/Post Condizioni)

- $\boldsymbol{caricaDaDB(tableName) \rightarrow D}$:
  - Precondizione: $tableName \neq \text{null} \land$ Tabella presente su DBMS MySQL con schema coerente e record ≥ 1
  - Postcondizione: $D.getNumberOfExamples() = |\{record \in tableName\}| \ \land \ D.getClassAttribute().isNumber() = \text{true}$
- $\boldsymbol{sort(D, A, inf, sup) \rightarrow D'}$:
  - Precondizione: $D \neq \text{null} \land A \in D.explanatorySet \land 0 \leq inf \leq sup < D.getNumberOfExamples()$
  - Postcondizione: D' è una permutazione degli esempi di D per cui:
    $\forall k \in [inf, sup - 1]: D'.getExplanatoryValue(k, A.getIndex()) \leq D'.getExplanatoryValue(k + 1, A.getIndex()) \ \land$
    $(\forall k \notin [inf, sup]: D'.data[k] = D.data[k])$
- $\boldsymbol{getClassValue(D, i) \rightarrow y}$:
  - Precondizione: $0 \leq i < D.getNumberOfExamples()$
  - Postcondizione: $y = D.data[i].get(D.explanatorySet.size())$

c) Tabella Costruttori × Osservazioni per `Data`

Costruttori ↓ / Osservazioni →	$getNumberOfExamples(D)$	$getClassValue(D, i)$	$getExplanatoryValue(D, i, j)$
$caricaDaDB(tableName)$	N (totale righe lette da SQL)	Valore numerico dell'ultima colonna del record i	Valore del record i alla colonna j
$sort(D, A, inf, sup)$	$getNumberOfExamples(D)$	$getClassValue(perm(D), i)$	$getExplanatoryValue(perm(D), i, j)$ con intervallo $[inf, sup]$ ordinato rispetto ad A

5.3 Specifica Formale di `SplitNode`

a) Specifica Sintattica (Sorts e Operazioni)

- **Sorts**: `SplitNode`, `Data`, `Integer`, `Attribute`, `Double`, `String`, `Object`, `SplitInfo`
- **Costruttori**:
  - $creaDiscreteNode: Data \times Integer \times Integer \times DiscreteAttribute \to SplitNode$
  - $creaContinuousNode: Data \times Integer \times Integer \times ContinuousAttribute \to SplitNode$
- **Osservazioni**:
  - $getAttribute: SplitNode \to Attribute$
  - $getVariance: SplitNode \to Double$
  - $getNumberOfChildren: SplitNode \to Integer$
  - $getSplitInfo: SplitNode \times Integer \to SplitInfo$
  - $testCondition: SplitNode \times Object \to Integer$
  - $formulateQuery: SplitNode \to String$

b) Specifica Semantica Assiomatica (Pre/Post Condizioni)

- $\boldsymbol{testCondition(node, value) \to branchIndex}$:
  - Precondizione: $node \neq$ null $\wedge\ value \neq$ null
  - Postcondizione: $branchIndex \in [0, node.getNumberOfChildren() - 1]$ tale che la condizione di partizione associata al ramo $branchIndex$ sia soddisfatta da $value$, altrimenti -1 se nessun ramo è compatibile.
- $\boldsymbol{getVariance(node) \to \sigma^2}$:
  - Precondizione: $node \neq$ null
  - Postcondizione: $\sigma^2 = \sum_{k=0}^{getNumberOfChildren() - 1} SSE(trainingSet, splitInfo_k.begin, splitInfo_k.end)$

c) Tabella Costruttori × Osservazioni per `SplitNode`

Costruttori ↓ / Osservazioni →	$getNumberOfChildren(n)$	$testCondition(n, val)$	$getVariance(n)$
$creaDiscreteNode(D, b, e, A)$	$\lvert\{distinct(A) \in D[b..e]\}\rvert$	Indice k per cui $splitValue_k = val$, oppure -1	$\sum_{k=0}^{K-1} SSE(D, begin_k, end_k)$
$creaContinuousNode(D, b, e, A)$	2 (se taglio ottimo identificato), 1 (se costante)	0 se $val \leq \tau^*$; 1 se $val > \tau^*$; -1 alt.	$\min_\tau \left[SSE(D_{\leq \tau}) + SSE(D_{> \tau})\right]$

5.4 Specifica Formale di `RegressionTree`

a) Specifica Sintattica (Sorts e Operazioni)

- **Sorts**: `RegressionTree`, `Data`, `Integer`, `Double`, `String`, `Node`
- **Costruttori**:
  - $learnTree: Data \rightarrow RegressionTree$ *(Induzione top-down dai dati)*
  - $carica: String \rightarrow RegressionTree$ *(Deserializzazione binaria da file `.dmp`)*
- **Osservazioni**:
  - $getRoot: RegressionTree \rightarrow Node$
  - $isLeaf: RegressionTree \rightarrow Boolean$
  - $predictClass: RegressionTree \times ObjectStream \rightarrow Double$
  - $printRules: RegressionTree \rightarrow String$
  - $salva: RegressionTree \times String \rightarrow \emptyset$ *(Effetto collaterale di serializzazione I/O)*

b) Specifica Semantica Assiomatica (Pre/Post Condizioni)

- $\boldsymbol{learnTree(trainingSet) \rightarrow T}$:
  - Precondizione: $trainingSet \neq \text{null} \wedge trainingSet.getNumberOfExamples() > 0$
  - Postcondizione: T è un albero binario/n-ario radicato sul nodo che minimizza l'errore quadratico totale:
    Se $|trainingSet| \leq 10\%|D_{tot}|$ allora T è radicato su un $LeafNode$ con valore predetto pari alla media locale $\bar{y} = \frac{1}{|S|}\sum y_i$;
    Altrimenti la radice è lo $SplitNode$ avente varianza aggregata minima tra tutti gli attributi esplicativi, e $childTree$ contiene i sottoalberi indotti ricorsivamente sulle partizioni generate.
- $\boldsymbol{carica(nomeFile) \rightarrow T}$:
  - Precondizione: $nomeFile$ indica un archivio di serializzazione valido e leggibile generato da $salva(...)$
  - Postcondizione: T è identico nello stato e nella topologia di nodi all'albero serializzato.

c) Tabella Costruttori × Osservazioni per `RegressionTree`

Costruttori ↓ / Osservazioni →	$isLeaf(T)$	$predictClass(T, input)$	$salva(T,f)/carica(f)$
$learnTree(D)$ **con** $\|D\| \leq$ threshold	true	$\bar{y} = \frac{1}{\|D\|}\sum_i y_i$	$carica(f) \equiv T$
$learnTree(D)$ **con** $\|D\| >$ threshold	false	Discesa ricorsiva in $childTree[root.testCondition(input)]$	$carica(f) \equiv T$
$carica(file)$	Valore del nodo radice deserializzato	Comportamento oracolare identico all'albero di origine	Consistenza idempotente dello stato serializzato

---

6. Diagrammi UML dei Package e delle Dipendenze (`«import»`)

Il sistema è ingegnerizzato secondo il pattern architetturale distribuito **Three-Tier (Architettura a Tre Livelli)**: 1. **Tier di Presentazione (Client)**: Eseguito sulla workstation utente, incapsulato nella classe `Client.Main` e supportato dalla libreria I/O `utility.Keyboard`. 2. **Tier Applicativo e Logica di Business (Server)**: Motore concorrente multi-thread (`MultiServer`, `ServerOneClient`), motore di induzione dell'albero (`tree`) e algebra dei dati (`data`). 3. **Tier di Persistenza e Dati (Database & Storage)**: DBMS relazionale MySQL 8.x per l'estrazione delle tabelle relazionali e filesystem per la persistenza serializzata `.dmp`.

```
flowchart TD
    subgraph Client_Module ["Modulo Maven Client"]
        P_CLIENT["package Client"]
        P_UTIL["package utility"]
    end

    subgraph Server_Module ["Modulo Maven Server"]
        P_SERVER["package server"]
        P_TREE["package tree"]
        P_DATA["package data"]
        P_DB["package database"]
    end

    subgraph External_Tier ["Tier di Persistenza Esterna"]
        MYSQL[("MySQL RDBMS (MapDB)")]
        FILESYSTEM[("Archivio File System (*.dmp)")]
    end

    %% Relazioni «import» esplicite
    P_CLIENT -->|"«import»"| P_UTIL
    P_SERVER -->|"«import»"| P_TREE
    P_SERVER -->|"«import»"| P_DATA
    P_TREE -->|"«import»"| P_DATA
    P_DATA -->|"«import»"| P_DB

    %% Anomalia Architetturale Ciclica Evidenziata
    P_TREE -.->|"«import» (Anomalia ADP: UnknownValueException)"| P_SERVER

    %% Canali di Rete e Persistenza
    P_CLIENT <==>|"TCP Socket (Porta 8080)<br/>Object Serialization Streams"| P_SERVER
    P_DB <==>|"JDBC / MySQL Protocol (Porta 3306)"| MYSQL
    P_TREE <==>|"Java Object Streams I/O"| FILESYSTEM

    classDef pkg fill:#e1f5fe,stroke:#0288d1,stroke-width:2px;
    classDef anomaly stroke:#d32f2f,stroke-width:2px,stroke-dasharray: 5 5;
    class P_CLIENT,P_UTIL,P_SERVER,P_TREE,P_DATA,P_DB pkg;
```

Proprietà Teoriche delle Dipendenze tra Package: 1. **Non-Transitività dell'Importazione**: L'importazione in Java non è transitiva: il fatto che `server` importi `tree` e `tree` importi `data` non attribuisce a `server` alcuna visibilità implicita su `data`. Ciascun package deve dichiarare le proprie dipendenze esplicite. 2. **Disaccoppiamento Assoluto di `Client`**: Il modulo `Client` non possiede alcuna dipendenza di compilazione o di linking verso `tree`, `data` o `database`. La comunicazione avviene esclusivamente tramite socket TCP su flusso binario serializzato, consentendo l'evoluzione e il rilascio disgiunto dei due moduli. 3. **Punto Fisso del Package `database`**: Il package `database` risiede alla base della gerarchia interna del server: non importa alcun package applicativo sovrastante, fungendo da componente a riuso puro.

---

7. Diagrammi UML delle Classi Dettagliati

La modellazione del diagramma delle classi adotta la notazione standard UML accademica: - Visibilità: `+` (public), `-` (private), `#` (protected), `~` (package-private). - Classi astratte e metodi astratti: contrassegnati con `<<abstract>>` e asterisco `*`. - Membri statici: contrassegnati con `$`. - Classe attiva: `ServerOneClient` contrassegnata con `<<active>>` e bordo rinforzato. - Inner class: relazione di contenimento ed esistenza annidata `SplitNode +-- SplitInfo`.

```
classDiagram
    %% PACKAGE DATA
    class Attribute {
        <<abstract>>
        -String name
        -int index
        #Attribute(name: String, index: int)
        +getName() String
        +getIndex() int
        +toString() String
    }

    class ContinuousAttribute {
        +ContinuousAttribute(name: String, index: int)
    }

    class DiscreteAttribute {
        -Set~String~ values
        +DiscreteAttribute(name: String, index: int, values: String[])
        +getNumberOfDistinctValues() int
        +iterator() Iterator~String~
    }

    class Data {
        -List~Example~ data
        -int numberOfExamples
        -List~Attribute~ explanatorySet
        -ContinuousAttribute classAttribute
        +Data(tableName: String)
        -isNumericType(sqlType: int) boolean
        +getNumberOfExamples() int
        +getNumberOfExplanatoryAttributes() int
        +getClassValue(exampleIndex: int) Double
        +getExplanatoryValue(exampleIndex: int, attributeIndex: int) Object
        +getExplanatoryAttribute(index: int) Attribute
        +getClassAttribute() ContinuousAttribute
        +toString() String
        +sort(attribute: Attribute, beginExampleIndex: int, endExampleIndex: int) void
        -swap(i: int, j: int) void
        -partition(attribute: DiscreteAttribute, inf: int, sup: int) int
        -partition(attribute: ContinuousAttribute, inf: int, sup: int) int
        -quicksort(attribute: Attribute, inf: int, sup: int) void
        +main(args: String[])$ void
    }

    class TrainingDataException {
        +TrainingDataException(message: String)
        +TrainingDataException(message: String, cause: Throwable)
    }

    %% PACKAGE DATABASE
    class Column {
        -String name
        -String type
        ~Column(name: String, type: String)
        +getColumnName() String
        +isNumber() boolean
        +toString() String
    }

    class Example {
        -List~Object~ example
        +add(o: Object) void
        +get(i: int) Object
        +compareTo(ex: Example) int
        +iterator() Iterator~Object~
        +toString() String
    }

    class TableSchema {
        -List~Column~ tableSchema
        +TableSchema(db: DbAccess, tableName: String)
        +getNumberOfAttributes() int
        +getColumn(index: int) Column
        +iterator() Iterator~Column~
```

```
}

class TableData {
    -DbAccess db
    +TableData(db: DbAccess)
    +getTransazioni(table: String) List~Example~
    +getDistinctColumnValues(table: String, column: Column) Set~Object~
}

class DbAccess {
    -String DRIVER_CLASS_NAME
    -String DBMS
    -String SERVER
    -String DATABASE
    -int PORT
    -String USER_ID
    -String PASSWORD
    -Connection conn
    +initConnection() void
    +getConn() Connection
    +getConnection() Connection
    +closeConnection() void
}

class DatabaseConnectionException {
    +DatabaseConnectionException(message: String)
}

class EmptySetException {
    +EmptySetException(message: String)
}

%% PACKAGE TREE
class RegressionTree {
    -Node root
    -RegressionTree[] childTree
    -RegressionTree()
    +RegressionTree(trainingSet: Data)
    -learnTree(trainingSet: Data, begin: int, end: int, numberOfExamplesPerLeaf: int) void
    -isLeaf(trainingSet: Data, begin: int, end: int, numberOfExamplesPerLeaf: int) boolean
    -determineBestSplitNode(trainingSet: Data, begin: int, end: int) SplitNode
    +printRules() void
    -printRules(current: String) void
    +printTree() void
    +toString() String
    +predictClass(out: ObjectOutputStream, in: ObjectInputStream) Double
    +salva(nomeFile: String) void
    +carica(nomeFile: String)$ RegressionTree
}

class Node {
    <<abstract>>
    -int idNodeCount$
    -int idNode
    -int beginExampleIndex
    -int endExampleIndex
    -double variance
    ~Node(trainingSet: Data, beginExampleIndex: int, endExampleIndex: int)
    ~getIdNode() int
    ~getBeginExampleIndex() int
    ~getEndExampleIndex() int
    ~getVariance() double
    ~getNumberOfChildren()* int
    +toString() String
}

class LeafNode {
    -Double predictedClassValue
    ~LeafNode(trainingSet: Data, beginExampleIndex: int, endExampleIndex: int)
    ~getPredictedClassValue() Double
    ~getNumberOfChildren() int
    +toString() String
}

class SplitNode {
    <<abstract>>
```

```
    -Attribute attribute
    #List~SplitInfo~ mapSplit
    -double splitVariance
    ~SplitNode(trainingSet: Data, beginExampleIndex: int, endExampleIndex: int, attribute: Attribute)
    #setSplitInfo(trainingSet: Data, beginExampleIndex: int, endExampleIndex: int, attribute: Attribute)* void
    ~testCondition(value: Object)* int
    ~getAttribute() Attribute
    ~getVariance() double
    ~getNumberOfChildren() int
    ~getSplitInfo(child: int) SplitInfo
    ~formulateQuery() String
    +compareTo(o: SplitNode) int
    +toString() String
}

class SplitInfo {
    -Object splitValue
    -int beginIndex
    -int endIndex
    -int numberChild
    -String comparator
    ~SplitInfo(splitValue: Object, beginIndex: int, endIndex: int, numberChild: int)
    ~SplitInfo(splitValue: Object, beginIndex: int, endIndex: int, numberChild: int, comparator: String)
    ~getBeginIndex() int
    ~getEndIndex() int
    ~getSplitValue() Object
    ~getNumberChild() int
    ~getComparator() String
    +toString() String
}

class DiscreteNode {
    ~DiscreteNode(trainingSet: Data, beginExampleIndex: int, endExampleIndex: int, attribute: DiscreteAttribute)
    #setSplitInfo(trainingSet: Data, beginExampleIndex: int, endExampleIndex: int, attribute: Attribute) void
    ~testCondition(value: Object) int
    +toString() String
}

class ContinuousNode {
    ~ContinuousNode(trainingSet: Data, beginExampleIndex: int, endExampleIndex: int, attribute: ContinuousAttribute)
    #setSplitInfo(trainingSet: Data, beginExampleIndex: int, endExampleIndex: int, attribute: Attribute) void
    ~testCondition(value: Object) int
    +toString() String
}

%% PACKAGE SERVER
class MultiServer {
    -int PORT
    +MultiServer(port: int)
    -run() void
}

class ServerOneClient {
    <<active>>
    -Socket socket
    -ObjectInputStream in
    -ObjectOutputStream out
    +ServerOneClient(s: Socket)
    +run() void
}

class UnknownValueException {
    +UnknownValueException(message: String)
}

%% PACKAGE CLIENT & UTILITY
class ClientMain {
    +main(args: String[])$ void
}

class Keyboard {
    -boolean printErrors$
    -int errorCount$
    -String current_token$
    -StringTokenizer reader$
    -BufferedReader in$
```

```
        +getErrorCount()$ int
        +resetErrorCount(count: int)$ void
        +getPrintErrors()$ boolean
        +setPrintErrors(flag: boolean)$ void
        -error(str: String)$ void
        -getNextToken()$ String
        -getNextToken(skip: boolean)$ String
        -getNextInputToken(skip: boolean)$ String
        +endOfLine()$ boolean
        +readString()$ String
        +readWord()$ String
        +readBoolean()$ boolean
        +readChar()$ char
        +readInt()$ int
        +readLong()$ long
        +readFloat()$ float
        +readDouble()$ double
    }

    %% RELAZIONI DI GENERALIZZAZIONE (is-a)
    Attribute <|-- ContinuousAttribute
    Attribute <|-- DiscreteAttribute
    Node <|-- LeafNode
    Node <|-- SplitNode
    SplitNode <|-- DiscreteNode
    SplitNode <|-- ContinuousNode
    Exception <|-- TrainingDataException
    Exception <|-- DatabaseConnectionException
    Exception <|-- EmptySetException
    Exception <|-- UnknownValueException
    Thread <|-- ServerOneClient

    %% REALIZZAZIONI DI INTERFACCE
    Iterable <|.. DiscreteAttribute
    Iterable <|.. Example
    Comparable <|.. Example
    Iterable <|.. TableSchema
    Comparable <|.. SplitNode
    Serializable <|.. Attribute
    Serializable <|.. Node
    Serializable <|.. SplitInfo
    Serializable <|.. RegressionTree

    %% INNER CLASS / NESTING
    SplitNode +-- SplitInfo

    %% COMPOSIZIONE (rombo pieno: ciclo di vita vincolato)
    RegressionTree *-- "1" Node : root
    RegressionTree *-- "0..*" RegressionTree : childTree
    SplitNode *-- "1..*" SplitInfo : mapSplit
    Data *-- "0..*" Example : data
    Data *-- "1..*" Attribute : explanatorySet
    Data *-- "1" ContinuousAttribute : classAttribute
    TableSchema *-- "1..*" Column : tableSchema

    %% AGGREGAZIONE (rombo vuoto: riferimento debole)
    TableData o-- "1" DbAccess : db
    SplitNode o-- "1" Attribute : attribute

    %% DIPENDENZE / ASSOCIAZIONI
    MultiServer ..> ServerOneClient : spawns thread
    ServerOneClient ..> RegressionTree : uses
    ServerOneClient ..> Data : uses
    ClientMain ..> Keyboard : uses for I/O
```

---

8. Diagramma di Sequenza UML del Protocollo Socket

Il sequence diagram descrive l'interazione sincrona su socket TCP (porta 8080) tra il client a console e il thread attivo `ServerOneClient`, dettagliando i tre casi d'uso principali del protocollo didattico: 1. **Comando** `0`: Connessione al DBMS MySQL e caricamento in memoria

del dataset di training. 2. **Comando `1`** : Esecuzione dell'algoritmo di apprendimento `learnTree(...)` e persistenza binaria dell'albero serializzato su disco (`.dmp`). 3. **Comando `3`** : Predizione interattiva guidata: il server formula le query per ciascun ramo di split, il client acquisisce la risposta dell'utente, e la foglia produce infine la predizione numerica continua.

```
sequenceDiagram
    autonumber
    actor Utente as Utente Terminale
    participant Client as Client.Main
    participant Server as ServerOneClient (Thread Attivo)
    participant Data as Data (TrainingSet)
    participant DB as DbAccess / MySQL
    participant Tree as RegressionTree

    Note over Client, Server: Connessione Socket TCP stabilita su porta 8080

    %% FASE 1: CARICAMENTO DATI (COMANDO 0)
    Utente->>Client: Seleziona [1] Apprendi da dati + inserisce tabella "provaC"
    Client->>Server: out.writeObject(0)
    Client->>Server: out.writeObject("provaC")
    Server->>Data: new Data("provaC")
    Data->>DB: initConnection() & SELECT * FROM `provaC`
    DB-->>Data: Recordset & Metadati Colonne
    Data-->>Server: Istanza trainingSet allocata nello Heap
    Server->>Client: out.writeObject("OK")

    %% FASE 2: APPRENDIMENTO E SERIALIZZAZIONE (COMANDO 1)
    Client->>Server: out.writeObject(1)
    Server->>Tree: new RegressionTree(trainingSet)
    Note over Tree: learnTree(...) esegue Quicksort ricorsivo,<br/>minimizza SSE e genera nodi di split e foglie
    Tree-->>Server: RegressionTree completato
    Server->>Tree: salva("provaC.dmp")
    Note over Tree: Scrittura su ObjectOutputStream di provaC.dmp
    Tree-->>Server: File salvato con successo
    Server->>Client: out.writeObject("OK")

    %% FASE 3: PREDIZIONE INTERATTIVA (COMANDO 3)
    Utente->>Client: Desidera effettuare una predizione
    Client->>Server: out.writeObject(3)
    Server->>Tree: predictClass(out, in)

    loop Visita guidata attraverso i nodi interni di split
        Tree->>Client: out.writeObject("QUERY")
        Tree->>Client: out.writeObject(formulateQuery())
        Client->>Utente: Visualizza opzioni del test (es. "0:X=A", "1:X=B")
        Utente->>Client: Inserisce valore selezionato (es. 0)
        Client->>Tree: out.writeObject(0)
    end

    Note over Tree: Raggiunta la foglia: estrae predictedClassValue
    Tree-->>Server: Restituisce valore numerico continuo stimato
    Server->>Client: out.writeObject("OK")
    Server->>Client: out.writeObject(predictedClassValue)
    Client->>Utente: Stampa a video "Predicted class: 1.5"
```

---

9. Diagramma di Deployment Three-Tier

Il diagramma di deployment modella la topologia fisica dell'architettura Three-Tier, specificando i nodi hardware, i runtime Java SE 25, i processi in esecuzione e i canali di connessione di rete e di I/O locale:

```
flowchart TB
    subgraph Client_Node ["Node: Client Workstation [OS: Windows / Linux / macOS]"]
        subgraph Client_JVM ["Execution Environment: Java SE 25 Runtime (Client)"]
            subgraph Client_Artifact ["Artifact: Client.jar"]
                C_MAIN["Main (Processo Console)"]
                C_UTIL["utility.Keyboard"]
            end
        end
    end

    subgraph Server_Node ["Node: Application Server [OS: Linux / Windows Server]"]
        subgraph Server_JVM ["Execution Environment: Java SE 25 Runtime (Server)"]
            subgraph Server_Artifact ["Artifact: Server.jar"]
                S_MULTI["server.MultiServer (Porta 8080)"]
                S_ONE["server.ServerOneClient (Thread Concorrenti)"]
                S_CORE["tree.RegressionTree & data.Data Engine"]
            end
        end
        subgraph Server_Disk ["Local Persistent File Storage"]
            DMP_STORE[("Archivi *.dmp Serializzati<br/>(es. provaC.dmp)")]
        end
    end

    subgraph DB_Node ["Node: Database Server [OS: Linux Server]"]
        subgraph DBMS_Instance ["DBMS: MySQL Server 8.x (Porta 3306)"]
            subgraph DB_Schema ["Database: MapDB"]
                TBL_PROVAC[("Tabella provaC")]
                TBL_SERVO[("Tabella servo")]
            end
        end
    end

    C_MAIN <==>|"TCP/IP Socket Protocol (Porta 8080)<br/>Object Serialization Stream"| S_MULTI
    S_MULTI -.->|"Spawns"| S_ONE
    S_ONE <==>|"In-Memory Method Invocations"| S_CORE
    S_CORE <==>|"File System I/O<br/>ObjectOutputStream / ObjectInputStream"| DMP_STORE
    S_CORE <==>|"JDBC / MySQL Protocol (Porta 3306)<br/>MySQL Connector/J"| DBMS_Instance

    classDef nodeBox fill:#eceff1,stroke:#455a64,stroke-width:2px;
    classDef compBox fill:#e8f5e9,stroke:#2e7d32,stroke-width:1.5px;
    class Client_Node,Server_Node,DB_Node nodeBox;
    class Client_JVM,Server_JVM,DBMS_Instance compBox;
```

---

10. Modellazione dei Nuovi Package e Componenti dell'Estensione (Fasi 1–6)

L'evoluzione architetturale del sistema introduce due nuovi package e un set completo di componenti per la comunicazione distribuita via HTTP/REST e l'interfaccia utente web reattiva.

10.1 Diagramma delle Classi: Package `dto` (Contratti JSON Immutabili)

I Data Transfer Object (DTO) sono realizzati come **Java Records** immutabili, incapsulando lo stato senza boilerplate e garantendo trasparenza referenziale e thread-safety intrinseca:

```
classDiagram
    class Serializable {
        <<interface>>
    }

    class TreeRequestDto {
        <<record>>
        -String tableName
        +tableName() String
    }

    class PredictionStepDto {
        <<record>>
        -boolean isLeaf
        -Double predictedValue
        -String attributeName
        -List~String~ branchOptions
        +isLeaf() boolean
        +predictedValue() Double
        +attributeName() String
        +branchOptions() List~String~
    }

    class TreeResponseDto {
        <<record>>
        -String tableName
        -String treeRules
        -PredictionStepDto rootStep
        +tableName() String
        +treeRules() String
        +rootStep() PredictionStepDto
    }

    class PredictRequestDto {
        <<record>>
        -String tableName
        -List~Integer~ path
        +tableName() String
        +path() List~Integer~
    }

    Serializable <|.. TreeRequestDto
    Serializable <|.. PredictionStepDto
    Serializable <|.. TreeResponseDto
    Serializable <|.. PredictRequestDto

    TreeResponseDto o-- PredictionStepDto : aggrega rootStep
```

10.2 Diagramma delle Classi: Componenti Spring Boot REST e Vaadin Flow

```
classDiagram
    direction TB

    %% LATO SERVER (Spring Boot REST)
    class ServerApplication {
        +main(args: String[]) void$
    }

    class TreeController {
        -TreeService treeService
        +TreeController(treeService: TreeService)
        +learn(request: TreeRequestDto) ResponseEntity~TreeResponseDto~
        +load(request: TreeRequestDto) ResponseEntity~TreeResponseDto~
        +predict(request: PredictRequestDto) ResponseEntity~PredictionStepDto~
    }

    class TreeService {
        -Map~String, RegressionTree~ activeTrees
        +learnFromDb(tableName: String) TreeResponseDto
        +loadFromFile(tableName: String) TreeResponseDto
        +predictStep(tableName: String, path: List~Integer~) PredictionStepDto
        +clearCache() void
    }

    class GlobalExceptionHandler {
        +handleFileNotFound(ex: FileNotFoundException) ResponseEntity~Map~
        +handleDomainExceptions(ex: Exception) ResponseEntity~Map~
        +handleIoAndFormatExceptions(ex: Exception) ResponseEntity~Map~
        +handleGenericException(ex: Exception) ResponseEntity~Map~
    }

    %% LATO CLIENT (Vaadin Flow)
    class ClientApplication {
        +main(args: String[]) void$
    }

    class TreeRestClientService {
        -RestClient restClient
        -ObjectMapper OBJECT_MAPPER$
        +TreeRestClientService(restClientBuilder: RestClient.Builder, baseUrl: String)
        +TreeRestClientService(baseUrl: String)
        +learnTree(tableName: String) TreeResponseDto
        +loadTree(tableName: String) TreeResponseDto
        +predictStep(tableName: String, path: List~Integer~) PredictionStepDto
    }

    class MainView {
        -TreeRestClientService restService
        -List~Integer~ currentPath
        -String currentTableName
        +MainView(restService: TreeRestClientService)
        -buildHeader() Component
        -buildConfigurationSection() Component
        -buildPredictionSection() Component
        -renderStep(step: PredictionStepDto) void
    }

    %% RELAZIONI SERVER
    ServerApplication ..> TreeController : configura via @Import
    ServerApplication ..> TreeService : configura via @Import
    ServerApplication ..> GlobalExceptionHandler : configura via @Import
    TreeController o-- TreeService : dependency injection
    TreeController ..> GlobalExceptionHandler : intercetta eccezioni

    %% RELAZIONI CLIENT
    ClientApplication ..> MainView : avvia UI Vaadin Flow
    MainView o-- TreeRestClientService : dependency injection

    %% RELAZIONE DI COMUNICAZIONE HTTP/JSON TRA CLIENT E SERVER
    TreeRestClientService ..> TreeController : chiama via HTTP REST (JSON)
```

10.3 Analisi dei Principi di Ingegneria del Software e OOP Avanzata

1. **Java Records come Value Objects e Tipi Astratti Puri:**
   - I DTO dell'estensione sono definiti come `record` introdotti in Java moderno. A differenza delle classi JavaBean tradizionali, i record sono classi immutabili i cui campi sono `private final`, prevenendo aliasing bug e modifiche di stato indesiderate durante il transito tra thread concorrenti e livelli applicativi.
   - Forniscono implementazioni canoniche di `equals()` e `hashCode()` basate sul valore di tutti i componenti (structural equality), perfettamente aderenti alla definizione di Value Object del Domain-Driven Design.
2. **Inversion of Control (IoC) e Dependency Injection (DI):**
   - Sia nel backend che nel frontend web, l'accoppiamento tra componenti è gestito tramite iniezione delle dipendenze via costruttore (`TreeController(TreeService)` e `MainView(TreeRestClientService)`).
   - Questo principio disaccoppia la creazione delle istanze dal loro utilizzo, facilitando l'isolamento dei test (es. mock del server o del client) e garantendo il rispetto del *Single Responsibility Principle* (SRP).
3. **Layered Architecture e Separation of Concerns (SoC):**
   - **Presentation Layer (Web UI):** `MainView` si occupa esclusivamente dell'ergonomia visiva, dei layout e dell'acquisizione degli eventi utente, delegando la logica di rete al client service.
   - **Client Communication Layer:** `TreeRestClientService` incapsula gli endpoint HTTP, la serializzazione JSON e la traduzione dei codici di stato di errore in eccezioni applicative.
   - **REST Controller Layer:** `TreeController` gestisce il protocollo HTTP, espone i contratti OpenAPI e orchestra le risposte con codici semantici (200, 400, 404).
   - **Application Service Layer:** `TreeService` gestisce la concorrenza via `ConcurrentHashMap` e funge da ponte tra le richieste DTO e il core matematico.
   - **Domain Core Layer:** `RegressionTree` e `Data` restano totalmente agnostici rispetto al protocollo di trasporto (funzionano identicamente sia sotto socket TCP che sotto REST API).
4. **Pattern Stateless e Scalabilità:**
   - La memorizzazione del cammino (`path: List<Integer>`) sul client e la sua valutazione istantanea tramite `RegressionTree.navigate(path)` elimina la necessità di mantenere thread bloccati in ascolto per ciascun client connesso, permettendo al server di scalare a migliaia di utenti simultanei.
5. **Component-Based UI in Puro Java (Vaadin Flow):**
   - L'interfaccia utente è strutturata ad oggetti secondo una gerarchia ad albero di componenti (Composite Pattern), dove layout complessi (`VerticalLayout`, `HorizontalLayout`) incapsulano widget atomici (`Button`, `TextField`, `TextArea`, `RadioButtonGroup`), gestiti tramite listener polimorfici orientati agli eventi.

10.4 Diagramma di Sequenza: Flusso REST Stateless Completo

```
sequenceDiagram
    autonumber
    actor Utente as Utente Web (Browser)
    participant UI as MainView (Vaadin :8081)
    participant ClientService as TreeRestClientService
    participant Controller as TreeController (:8080)
    participant Service as TreeService
    participant Tree as RegressionTree
    participant DB as MySQL (MapDB)

    %% APPRENDIMENTO
    Utente->>UI: Seleziona "Database" e inserisce "provaC"
    UI->>ClientService: learnTree("provaC")
    ClientService->>Controller: POST /api/trees/learn {"tableName":"provaC"}
    Controller->>Service: learnFromDb("provaC")
    Service->>DB: SELECT su provaC
    DB-->>Service: Recordset 15 tuple
    Service->>Tree: new RegressionTree(data)
    Note over Tree: Induzione dell'albero e calcolo SSE
    Service->>Tree: salva("provaC.dmp")
    Service->>Tree: navigate(List.of())
    Tree-->>Service: PredictionStepDto (radice: attribute="X", options=["=A","=B"])
    Service-->>Controller: TreeResponseDto
    Controller-->>ClientService: HTTP 200 OK (JSON)
    ClientService-->>UI: TreeResponseDto
    UI->>Utente: Visualizza regole e primo bivio [=A] / [=B]

    %% PREDIZIONE PASSO 1
    Utente->>UI: Clicca su opzione 1 ([=B])
    UI->>ClientService: predictStep("provaC", [1])
    ClientService->>Controller: POST /api/trees/predict {"tableName":"provaC", "path":[1]}
    Controller->>Service: predictStep("provaC", [1])
    Service->>Tree: navigate([1])
    Note over Tree: Raggiunta foglia per X=B
    Tree-->>Service: PredictionStepDto (isLeaf=true, predictedValue=10.0)
    Service-->>Controller: PredictionStepDto
    Controller-->>ClientService: HTTP 200 OK (JSON)
    ClientService-->>UI: PredictionStepDto
    UI->>Utente: Badge "Classe Predetta: 10.0"
```

---

11. Conclusioni e Valutazione di Qualità Ingegneristica

L'analisi architetturale e teorica condotta sul sistema **Albero di Regressione** certifica l'eccellenza e la conformità ingegneristica della soluzione rispetto ai canoni dell'Ingegneria del Software avanzata: 1. **Coerenza con la Teoria dell'Astrazione**: La mappatura delle 6 classi sintattiche, il rispetto della cittadinanza di prima classe dei valori e la demarcazione tra specifica e realizzazione assicurano un'architettura manutenibile, testabile ed estendibile. 2. **Robustezza dell'Incapsulamento**: L'impiego sinergico di pareti opache e traslucide tutela la consistenza interna dei dati impedendo manomissioni accidentali, mentre la separazione tra Specification (`RegressionTree`) e Body (`Node` , `SplitNode`) concretizza in ambiente Java il rigore formale dei moduli Ada. 3. **Flessibilità Evolutiva**: La scelta consapevole del paradigma a oggetti per la gerarchia dei nodi e degli attributi, validata attraverso la matrice di William R. Cook, garantisce un'estensibilità aperta sui costruttori, rendendo l'infrastruttura nativamente pronta all'integrazione di nuovi operatori di split o modelli d'insieme (*ensemble methods*). 4. **Modernità dell'Estensione Distribuita**: L'integrazione delle Fasi 1–6 dimostra la naturale evoluzione del sistema da un modello socket accademico a un'infrastruttura di livello enterprise, introducendo contratti DTO immutabili, documentazione OpenAPI conforme agli standard industriali e un'interfaccia reattiva moderna.